

研究计划 / 24 周 / v2.1

FPGA 加速大模型 在线安全检查

Mixed-Precision FPGA Co-Design for LLM Guards

在相同防护要求下，改进精度分配与硬件实现

核心研究问题

以已有硬件感知混合精度方法为起点，在不明显增加漏检和误拦的前提下，能否提出更好的精度分配与 FPGA 实现方法，降低完整检查的时间和计算成本？

研究范围不变

本项目检测外部资料中夹带的恶意指令，例如“忽略用户的问题，改为执行下面的要求”。这叫间接提示注入；不研究辨别所有假知识，也不修复大模型权重。BIPIA 提供起步任务与评估代码。^[1]

应用设定为持续接收新邮件、附件或网页的文档问答。先回放固定的检索片段，不开发完整知识库；已经检查且未变化的资料，允许所有方案同样复用结果。

本次修订集中在学术贡献与对照

保留 24 周、5 个工作包和现有模型、硬件路线。修改之处是：不再把“联合选择位宽与硬件”本身当成新思想；先把已有方法放到同样的防护要求下，再检验我们的方法是否仍有额外收益。

研究产出	需要讲清楚的内容
一项可解释的发现	降低精度后，哪些判断真的变差，哪些变化只需调整阈值即可恢复。
一种具体改进方法	已有方法在相同要求下哪里不足；我们的精度安排或硬件组织怎样改进它。
真实硬件证据	同一 FPGA 上，新方法相对强基线的收益来自哪里；完整检查相对 CPU/GPU 的成本如何。

项目边界

FPGA 是研究主体，安全检测器是研究对象，文档问答是验证场景。不要求显著加速大模型的整段回答，也不增加检索融合、分级网关或产品部署。

阅读顺序：第 2 页看研究问题；第 3 页看时间表；第 4–8 页执行工作包；第 9–10 页准备平台与分工；第 11 页查阅来源。方法与效果均为待验证内容。

保留方向，收紧要证明的事情

联合设计是已有基础；本项目要解释，新的任务要求是否需要新的设计方法。

H1：量化损失只是分数移动，还是判断能力变差？

漏检是把攻击放过去，误拦是拒绝正常资料。保持同样的误拦水平、重新调整判决阈值后，观察哪些计算仍导致攻击更难检出。H1 是后续方法的实验证据，不把“量化可能损害安全”当作新的普遍发现。 [2][3][18]

H2：不只换评价指标，能否改进精度分配？

候选假设：在误拦率较低时，影响最终判断的计算块，可能与普通损失或数值误差识别出的重要部分不同。关键比较是：**已有方法也采用相同防护要求后，我们的具体选择方法是否仍然更好？** 低误拦检测率本身不是新指标。 [2]

H3：新的分配如何转化成真实硬件收益？

不再检验泛泛的“联合设计优于顺序设计”，而检验新增机制是否减少计算、数据搬运或等待。先按相同FPGA 资源预算比较，再解释位宽打包、共享引擎或缓存安排各自带来什么。 [6][16][17]

已有基础	对本项目意味着什么
HAQ、Q-BERT [4][5]	硬件感知与敏感性分配已有方法；不能只与统一位宽比较。
HAO、Quasar-ViT、FILM-QNN [16][17][6]	量化与 FPGA 设计结合已有先例；具体实现必须比合理优化的已有方案更好。
Critical Weight Protection [18]	为安全相关权重保留精度已有直接先例；独立检测器与硬件实现是任务差异，不自动等于新方法。
InfoQ、MixQuant [15][19]	全网影响和层间依赖已有人研究；重新评估候选全网只是起步工具，不作为原创点。

贡献的落点

争取证明一个具体不足，并提出对应的精度分配或硬件组织改进。若改善主要来自换用防护指标，就把结论限定为任务适配；不把“换应用+已有搜索”包装成新的通用方法。

02 / 总时间表

24 周工作包与研究里程碑

以实际启动日为第 1 周，按每周约 3–4 天投入安排。软件与小规模综合并行；设备等待另计。

工作包	第 1 月	第 2 月	第 3 月	第 4 月	第 5 月	第 6 月
WP1 · 任务与强基线	W1–3					
WP2 · 精度影响与成本	W3–4	W5–7				
WP3 · 方法改进		W6–8	W9–12			
WP4 · 硬件实现与归因			W10–12	W13–16	W17–18	
WP5 · 适用范围与写作					W18–20	W21–24

时间表不变，里程碑更加明确

节点	研究问题与交付结果
M1 · 第 3 周	我们在什么任务上检验什么改进？ 固定数据、主检测器和硬件路线；选定一个可对齐的强基线，列清哪些部分复现、哪些部分适配。
M2 · 第 7 周	精度降低会怎样改变防护？ 交付阈值调整后的敏感性结果、CPU/GPU 耗时分解及代表性算子的综合结果。
M3 · 第 12 周	改善来自评价标准，还是具体新方法？ 完成“已有方法—相同防护要求的已有方法—拟议方法”比较；给出方法说明和代表性硬件配置。
M4 · 第 18 周	收益是否在真实 FPGA 上成立？ 同板比较统一位宽、强适配基线和拟议方法；解释收益来自哪些计算或数据搬运变化。
M5 · 第 24 周	结论适用于哪些模型和负载？ 完成第二组任务或模型的有限验证、完整检查成本分析，以及论文初稿。

软件和硬件并行推进

第 3 周开始微调检测器，第 6 周开始代表性算子的综合，第 10 周启动主机接口与统一位宽硬件。WP3 的方法尚在调整时，这些公共部分仍可继续开发。

工作量保持原规模

六个月聚焦一个主要改进方法、一张 FPGA、一个主检测器。相关论文用于明确边界，不要求全部复现；第二组任务与第二模型二选一。第 22 周后集中补测与写作。

确定任务、模型与强基线

研究问题：现有方法适配到同样任务后，还留下什么值得改进的地方？

1. 固定任务与攻击入口

从 BIPIA 的 EmailQA 开始：用户问题正常，攻击者只能修改外部资料；首版只做英文、纯文本、单轮问答。整理 100 个正常／攻击配对任务，以无害且可判断的目标跑通记录。 [1]

先直接给定外部片段，研究检索之后的检查。各方案使用相同文本、顺序与缓存规则；这不是完整知识库投毒实验，不要求攻击者先突破内部权限。BIPIA 已归档，使用其固定版本即可。 [1]

2. 固定主检测器与效果参考

主模型沿用 BERT-Mini（4 层、隐藏维度 256），微调用于判断是否有注入；BERT-Tiny 仅用于早期调试。Prompt Guard 2 22M 是较强的独立参考，支持 512-token 窗口，使用前申请访问。 [7][2]

不从头训练语言模型。后续量化都与主检测器自己的浮点版本比较，另列它与强参考的差距。目标大模型固定为 Qwen3-4B-Instruct-2507，或版本可固定的 API；精度搜索不反复调用目标大模型。 [8]

3. 选一个主基线，不把文献表变成复现清单

先读 HAO、Quasar-ViT 的硬件选择方法，再结合 Q-BERT／InfoQ 判断怎样适配固定的 BERT-Mini。选其中最可对齐的一种作为主基线；明确保留哪些规则、修改哪些接口和任务指标，不额外训练视觉超网。

[16][17][5][15]

Critical Weight Protection 用于核对安全相关精度分配的已有思想，MixQuant 用于核对层间影响。二者是必须讨论的相关工作，不自动增加为完整实现任务。 [18][19]

需要固定的内容	保存方式
样本	原文、问题、正常答案、攻击标签、攻击类型、来源编号。
数据划分	训练、量化／搜索、最终阈值验证、测试集分开；同一来源及改写留在同组。
配置	模型和分词器版本、种子、位宽、阈值、设备信息及逐条输出。

M1 · 第 3 周

交付可运行小实验和一页方法对照表。明确主基线的适配边界、主模型和板卡；不以搭建完整 RAG 平台作为成果。

测清降低精度会改变什么

研究问题：哪些变化只是阈值移动，哪些是攻击和正常输入真的更难区分？

1. 建立浮点与统一位宽基线

完成主检测器微调后固定网络结构。建立浮点、W8A8（8 位权重／8 位激活）、W4A8 三组参考。归一化、softmax 和累加保留所需精度，记录各算子的格式和开销；不把 4 位权重称作整个网络都是 4 位。

2. 在相同误拦水平下比较

沿用约 1% 正常资料误拦率作为起步点，再看邻近工作点。各版本使用相同阈值选择规则；搜索用开发数据，最终配置在独立阈值验证集上重新定阈值并冻结，测试集只报告实际误拦、检出率及误差范围。

候选筛选暂允许攻击检出率相对同模型浮点参考下降不超过 1 个百分点；该容差在 WP1 固定。先用数百样本调试，最终尽量准备至少 3,000 条独立正常测试片段；同一资料的多个窗口不算独立来源。加入 NotInject 一类看起来像攻击、实际正常的资料。[3]

3. 找到值得解释的变化，而不只给层排序

按线性层或连续计算块分组，每次降低一组的权重位宽，记录新增漏检、误拦与分数变化。将这些结果与普通数值误差／损失排名比较，再对排名靠前的少数组合做联合实验。

重点回答：重新调阈值能恢复多少？哪些错误仍然留下？按原有重要性排名保留精度，是否已能解决问题？层间影响已有相关方法，本阶段的价值是找到本任务中具体、可重复的不足。[15][19]

4. 同时建立真实的成本参考

优化 CPU 线程、GPU 批量与推理路径，支持时使用 ONNX Runtime。记录分词、模型计算、数据往返。对代表性矩阵做 4/8 位、不同并行度和打包方式的综合，记录资源、频率与周期。[11]

共享计算引擎的物理资源不会随层自动减少。要测清低位宽带来的是更多并行乘法、更少搬运，还是仅仅缩小权重文件；把打包、转换与非矩阵运算的开销也算进去。

M2 · 第 7 周

交付三张图：阈值调整后的防护变化、各计算块的敏感性、软件／硬件成本分解。用一页说明指出最值得 WP3 解决的具体不足，而不是只写“量化影响安全”。

在强基线之上提出具体改进

研究问题：同样采用防护要求，为什么我们的精度分配或硬件组织会更好？

输入与输出

输入：固定检测器、开发数据、防护要求和板卡预算。输出：静态位宽表、共享引擎规模、并行／存储配置，以及它优于基线的原因。

1. 先做清楚三个关键对照

方案	评价与选择方式
A · 已有方法	保留其常规损失／误差目标，适配相同网络和硬件空间；最后仍报告防护表现。
B · 已有方法＋相同防护要求	在候选选择过程中就使用我们的误拦／检出要求，保留原有搜索和资源配置原则；不能只事后过滤。
C · 拟议改进	使用与 B 相同的防护要求、空间和硬件预算，只改变我们提出的选择或实现机制。

A→B 区分任务指标的作用；B→C 才检验具体方法的额外贡献。 候选评估预算、微调额度、阈值规则与硬件成本模型保持可比。基线的任务适配与完整原论文复现分别表述。

2. 起步搜索是工具，不预先认定为新算法

先按层或连续块选 4/8 位权重，激活固定 8 位；部署时精度固定。从 8 位方案尝试降低一组位宽，结合硬件估计保留少量候选，全网重新评分。仅对最终短名单做相同额度的量化微调，再判断是否满足要求。

这套搜索先用于定位问题。WP2 若显示某些判断对精度特别敏感，可据此改进恢复高精度的顺序或分组；若收益主要受打包与等待限制，就聚焦硬件组织。只选一个有证据支持的主要改进，不同时增加多个新模块。

3. 把改进落实到硬件可执行的配置

采用共享分块矩阵引擎，联合考虑整体引擎规模、低位宽打包模式、各层调度和缓存。计算完整检查成本，而不只优化参数数目。相同精度、相同数值结果只需评估一次检测能力；硬件候选可复用分数。

将拟议机制单独去掉做一次对照，检查 B→C 的改善是否来自它。顺序／联合设计可以继续作为解释性比较，但不再承担主要创新声明。 [6][16][17]

M3 · 第 12 周

交付 A/B/C 比较、方法说明、搜索程序与代表性配置。说明基线的具体不足和改进机制；据此冻结 WP4 的同板比较方案。

用 FPGA 检验收益从哪里产生

研究问题：相同防护要求下，拟议方法相对强基线的优势能否在板上实现？

部分	实现位置与职责
文本准备	主机 CPU 保留原文，完成分词，生成 token 编号、有效长度和注意力掩码。
主检测器	FPGA 完成词向量读取、编码器主要计算与分类；保留高精度的算子计入成本。
完成检查	返回风险分数、片段编号和状态；需要检查的片段完成后，才把对应资料交给大模型。
目标大模型	GPU 或 API；使用与软件对照相同的模型版本和输入。

W10–12：先完成公共硬件与统一位宽基线

建立主机链路和统一 8 位的分块矩阵引擎。记录有效计算、外存访问和等待；预算包含词向量、权重与中间结果。先测代表性层，再扩展到完整网络。

W13–15：实现由配置驱动的静态混合精度

加入 4/8 位数据打包、计算映射与配置生成，缩放规则与量化软件参考一致。attention、softmax、归一化等算子逐项落实，不能把 FINN 导出成功等同于完成 Transformer 板级实现。[9]

共享引擎以实际可用的打包和并行模式运行；无法转化为速度的位宽节省，按存储收益单独报告。改进机制和基线使用同等优化投入。

W16–18：保留三种主要板级实现

统一位宽方案、B：相同防护要求的强基线、C：拟议方法。三者使用同一张板、相同资源上限和目标时钟；报告实际频率、资源及达到的防护水平。A 和顺序设计主要作软件／综合对照，不自动增加第四次完整硬件实现。

选约 1,000 个片段做最终软硬件一致性核对，含短输入和最大长度。整数路径核对数值，近似算子按预定误差核对，并查看最终判决；开发期间只用小样本定位相关错误。

M4 · 第 18 周

交付三种主要实现的板级结果：完整检查时间、资源、外存流量和功耗。能说明 B→C 的收益来自哪些计算、打包、调度或搬运变化；“上板成功”不替代研究结论。

首版只做通过／拦截，不加入 GPU 复查分流。板端预装输入只用于内核计时，完整检查必须计入主机准备与数据往返。综合估计与板级实测分开保存。

验证适用范围，形成论文证据

研究问题：新方法解决了什么，适用于哪里；哪些收益只是任务适配？

1. 先证明方法，再比较平台

以同一 FPGA 上 B→C 的比较为核心，同时保留 A→B，区分评价标准和新机制的作用。再比较优化 CPU/GPU；不支持低位宽的设备用满足同等防护要求的最快实现，不拿慢速低位宽仿真作性能基线。

2. 外部验证保持有限

在未用于开发的资料与攻击上测试，并增加少量改写、位置变化和重复尝试：起步 20 个任务、每个最多 3 个变体。再选另一组文档任务或第二个小型编码器，沿用同一规则；第二模型可只做软件与综合，范围明确标注。

3. 接入目标大模型，但不让生成时间掩盖检查收益

测试正常／攻击资料、有／无检查四种组合，报告最终攻击成功率和正常任务完成率；拦截、超时都留在统计中。另测开始回答的等待、整段响应时间和服务吞吐。[12]

核心图表	需要回答的问题
① 精度影响	调阈值后哪些损失仍存在；普通重要性排名是否足够？
② A/B/C 防护—成本曲线	改善来自新的约束，还是提出的方法？
③ 方法拆分与同板证据	去掉主要改进后收益还剩多少；发生在计算还是搬运？
④ 完整检查延迟与能耗	分词、排队、传输和计算各占多少；同吞吐下成本如何？
⑤ 适用范围与问答效果	换长度、负载或任务后是否成立；最终回答有没有受益？

测量设置沿用，不增加开发负担

主长度为 128／256／512 tokens，批量 1／8／32，问答默认 5 个片段；超窗按同一规则分段。最终主配置预热后记录至少 10,000 次完成检查，重复 3 轮；开发和非关键组合用短测。功耗按同吞吐测量，整机与板卡分开。

M5 · 第 24 周

交付论文、原始数据与复现配置，围绕“具体不足 → 方法 → 公平对照 → 硬件证据”组织。未来更快模型的收益可单列推算；若主要改善来自任务适配，就按该结果定位，不扩大方法声明。

用现有平台起步

设施需求与 v2 保持一致；第一周确认可用性、接口、工具许可和远程运行权限。

设施	建议与用途	需要时间
Linux 工作站／服务器	约 12–24 个 CPU 核、64 GB 内存；大 FPGA 工程优先 128 GB。约 1 TB SSD 用于工具、模型和构建。	W1 起
NVIDIA GPU 一张	24 GB 显存是起步建议：微调检测器、GPU 对照、短上下文 4B 级目标模型。共卡并发按实际显存安排。	W2 起
FPGA 一张	沿用可稳定开发的 VEK280 或实验室 Alveo。先在 PL 做主方法，不把 AIE 或第二张板列为必做。	W1 确认 W6 综合 W10 上板
连接与调试条件	电源、散热、下载线、主机槽位或已有链路；确认 DMA、驱动、外存和工具许可。	W3 前
功耗采集	优先可记录的整机功率计；辅以 GPU／FPGA 遥测。只有板级读数时，只作板级能耗结论。	W6 接通 W18 正式测

VEK280 可以继续作为首选候选

官方资料将 VEK280 定位为基于 VE2802 的 AIE-ML 评估平台。本项目首先在可编程逻辑中验证位宽与资源分配；AIE 是否参与由算子和接口条件决定，不假设任意 4/8 位方案都能直接映射到 AIE。[10]

若主机链路尚未跑通，可先做板端内核测量并并行打通连接。已有成熟 Alveo 主机平台时直接沿用；本项目不要求 HBM，也不购置向量检索专用设备。

API 可用于问答，不能替代硬件性能测量

早期目标大模型可走 API，检测器仍在自己的机器运行。CPU/GPU 对照、共卡影响和能耗，需要可控制的本地或研究所服务器。大模型不放到 FPGA，也不从头训练。

设备安排服务于研究

先确认现有平台，第 6 周起并行做硬件。显存和内存是配置建议，不是兼容性保证；上下文、输出长度和并发按实际设备设置。

把可执行工作交给 Codex

保留研究所需的代码和实验，不增加产品网关、复杂审计或无关测试。

用途	工具与安排
训练与模型调用	Python、PyTorch、Transformers：微调小检测器，运行强参考及目标模型。
量化与方法比较	Brevitas+搜索脚本：统一管理 A/B/C 和硬件配置。原有逐块降位宽搜索保留为研究工具，不预定为论文新算法。[9]
FPGA 实现	Vivado/Vitis HLS；FINN/QONNX 用于支持的计算块，其余算子和主机接口单独实现。[9]
软件测速与分析	PyTorch、适用的 ONNX Runtime、设备事件与主机计时。正式计时关闭详细 Profiler，GPU 使用同步或设备事件。[11][13]
结果管理	Git、YAML、CSV/JSONL；模型、数值配置、数据划分和逐条结果保留。环境按训练/硬件流程分别固定版本。

Codex 负责实现、运行和整理，不只写脚本

在能访问 GPU、FPGA 工具和板卡的服务器上运行 Codex：委托数据准备、微调、量化、搜索、HLS/Tcl、仿真、综合、可远程执行的板测，以及图表和初稿。它应运行工具、读日志并修复相关错误。[14]

你负责确认研究问题与主基线、验收关键结果，以及账号授权和必须现场完成的设备操作。已完成工作继续复用；把执行提示中的方法与板级对照更新为本版 WP3/WP4 即可。

只保留影响实验结论的必要检查

检查数据分组和标签、指标/阈值计算、量化算术，以及最终软硬件一致性。开发用小样本；最终配置才做完整计时。**不增加哈希清单、重复完整性扫描、通用安全测试或 CI 平台。**改排版、画图不触发整套重跑。

第一周	行动与结果
第 1 天	确认主机、GPU、板卡、接口、工具；申请模型权限，建立项目。
第 2-3 天	整理 100 个配对任务、运行参考分数；阅读主基线和最接近的近期工作。
第 4-5 天	开始 BERT-Mini 微调与 W8A8 试验；留下任务配置、A/B/C 定义和第一张待画的图。

代码建议：data/、models/、methods/、hardware/、experiments/、results/。同一份配置驱动量化、综合与上板；真正缺少资源时，先完成不受阻的工作。

资料入口与本次变化

原论文、发布方和官方文档；查阅日期：2026-09-25。点击标题可打开来源。

[1] Microsoft · BIPIA

间接提示注入任务与评估代码；从 EmailQA 起步。仓库已归档，沿用固定版本即可。

[2] Meta · Llama Prompt Guard 2 22M

效果参考；核对 512-token 窗口、许可及完整模型规模。

[3] PiGuard · ACL 2025

误拦问题与 NotInject 正常文本；用于补充评估。

[4] HAQ · CVPR 2019

硬件感知混合精度；已有选择方法的背景。

[5] Q-BERT · AAAI 2020

BERT 的敏感性分析与低精度量化；精度分配对照。

[6] FILM-QNN · FPGA 2022

层内混合精度的 FPGA 实现；核对计算打包与资源利用的已有方法。

[7] Google Research · Smaller BERT Models

BERT-Mini、BERT-Tiny 等预训练模型与配置。

[8] Qwen · Qwen3-4B-Instruct-2507

固定的目标大模型；仅用于问答验证，不训练其权重。

[9] Brevitas / FINN 官方文档

量化建模和支持的硬件流程；另见 FINN 文档。

[10] AMD · VEK280 Evaluation Kit

核对 VE2802、AIE-ML、开发接口与板卡资料。

[11] ONNX Runtime · Quantization

优化 CPU/GPU 对照；核查实际支持的算子与数值格式。

[12] NVIDIA · LLM Benchmarking Metrics

区分开始回答时间、完整响应时间和吞吐量。

[13] PyTorch · Profiler

定位算子、内存和设备开销；正式计时关闭详细追踪。

[14] OpenAI · Codex CLI

读取项目、实现代码、执行工具和运行实验；需能访问实际计算设备。

[15] InfoQ · AAAI 2026

从全网信息变化衡量量化影响；不是只看单层局部误差。

[16] HAO · FCCM 2021

量化、网络与 FPGA 配置的协同优化；明确联合设计的已有基础。

[17] Quasar-ViT · ICS 2024

Transformer 混合精度、硬件成本建模与真实 FPGA 实现。

[18] Critical Weight Protection · ACL Findings 2026

完整标题：Preserving Fairness and Safety in Quantized LLMs Through Critical Weight Protection。保护生成模型的关键权重，不是独立注入检测器。

[19] MixQuant · 2026 年 7 月预印本

Adaptive Mixed-Precision Quantization for Large Language Models；研究上游位宽如何影响层敏感性。

v2.1 相对 v2.0 的局部修订

新增 HAO、Quasar-ViT、Critical Weight Protection 与 MixQuant 的对照；重写 H2/H3 与 WP3 的创新表述；同板主比较改为统一位宽、强适配基线和拟议方法。24 周、模型选择、4/8 位范围和设备需求保持不变。

文献边界：上述工作分别覆盖联合设计、安全相关精度保护或层间影响。本计划不承诺“首次”或某个性能提升；具体贡献由 A/B/C 及真实硬件结果确定。MixQuant 按预印本列示。